Coding Club Presentation Script

Introduction to Machine Learning and Neural Networks

Target time: about 20 minutes

Bracketed cues are section headers. The script is written to explain the ideas, not to narrate every code line.

0:00–1:15 | Welcome and the project's goal

[Notebook title && AI hierarchy image]

Hi, everyone. Today we're starting our introduction to machine learning and scientific computing. Scientific computing is everywhere, and we use it to solve both everyday problems as well as complex scientific challenges.

Scientific Computing: uses `mathematical models` and `numerical algorithms` to study and solve complex problems in science and engineering.

Examples include **Text Prediction** we use every day and Protein Structure Analysis, or AlphaFold, that discovers possible, previously unknown protein shapes.

We can see that both the cases rely on existing, or known, data or information, and subsequently use them to bring up something that's new. This is closely related to machine learning. However, before we dive into the field, we should take a look at what **machine learning** is and where it fits in the field of **artificial intelligence**.

As we can see in this illustration, a common misconception is that AI means chatbots or large language models. Instead, AI is the broad field. Machine learning is one approach within AI: instead of writing every rule by hand, we use examples to help a system learn a rule. Neural networks are one family of machine-learning models. Transformers are a type of neural-network architecture, and many large language models use transformers.

These ideas are similar to nested categories. While not every `LLM` has exactly the same design, they are not equal to `artificial intelligence` but rather belong to a hyper-specific field of it.

We are also not going to focus on these hyper specific fields. Instead, we will look at scientific computing and machine learning as a whole.

1:15–4:00 | What does it mean to learn?

[“What is Learning, really?”]

When we think about machine learning, many of us intuitively relate it to very deep and concrete examples that happened inside data centers or scientific laboratories. I want to bring a different perspective by looking at what is **learning**, exactly.

First, **observe examples**. In **supervised learning**, we collect examples of input, represented by an upper case `X`, and their answers, represented by a lower case `y`. In **unsupervised learning**, there are no supplied target answers, so methods look for structure in the inputs—for example, grouping customers with similar behavior.

For a supervised learning example, the price of the house may be related to its size and location. Therefore, we can use the size and location to estimate the house price, something we care about. In this case, `X` would be the size and location, while the `y` would be the price.

Second, **fit a pattern**. The learning algorithm uses those examples to choose a rule that connects X to y. It might discover that larger homes tend to cost more. Fitting means adjusting the model so its predictions are closer to the observed answers.

Third, **generalize**. We use the fitted rule on a new example the model has not seen before. A useful model should do more than remember its training examples; it should make reasonable predictions for new inputs.

This helps clarify what we mean when we say machine learning is not simply hard-coded. A program that stores examples and does something like `dataset.get(x)` is looking up a memorized answer. This only works for information that’s already in the dataset. For anything that’s previously unseen, the hard-coded program cannot offer an answer, while machine learning can.

4:00–5:00 | Common learning tasks

[“Common Tasks of Machine Learning.”]

There are two distinctions to keep in mind. One is what kind of answer we want. **Classification** predicts a category, like “spam” or “not spam.” **Regression** predicts a number, like a price or temperature.**Clustering** groups unlabeled data points based on “similarities”, without relying on pre-defined categories.

Classification and regression are common supervised tasks. Clustering is a common unsupervised task.

5:00–6:15 | Meet the stock-price data

[Show the data description and plot.]

Our example is a small Google stock dataset. The 2025 file contains nine monthly observations, from April through December. X is the month index, and y is a monthly price measure: the midpoint between the high and low on the month's first trading day.

We're asking a deliberately simple question: can we predict y using only the month index X? The scatter plot helps us see the data.

(Describe the data and the plot)

While not exactly, the data seems to be approximately linear, so a linear model may represent the connection between time and stock price.

6:15–8:00 | Fit and inspect a line

[Show the linear regression section and its graph.]

Let's first fit a straight line. A line is described by a slope and an intercept. The slope tells us how much the predicted price changes for each step in the month index. The intercept is the prediction when the index is zero.

The notebook calculates those values directly in plain Python using the least-squares formula. We don't need to read each line of code. The important idea is that the calculation chooses the line that fits the observed points according to that method.

[Point to the fitted line.]

The fitted line is approximately $\hat{y} = 19.88X + 131.61$. The hat on y means "predicted y." Notice that we're using a simple model and a single input. This makes it easy to understand, but it also limits what the model can represent.

8:00–9:15 | Residuals, loss, and accuracy

[Show the residuals and loss section.]

A **residual** is the difference between an actual observation and the model's prediction. If a point is above the line, its residual is positive; if it's below, the residual is negative.

A **loss** summarizes prediction errors with a number. Here we use mean squared error: square each residual and average the results. Larger misses count more because the errors are squared.

We can see the residual plots. If any of you have taken AP Stats, you will know that a residual plot with a pattern is bad. The residual plot of our linear regression shows a curved pattern, so maybe we could try another model, to further decrease the residual, or the loss.

9:15–10:30 | Try an exponential curve

[The exponential fit and its graph.]

A straight line may not capture a pattern that rises more quickly over time, so we also try an exponential model. The notebook fits it by taking the natural logarithm of the observed prices, fitting a line to those transformed values, and then converting the result back to the price scale. The plot samples the fitted formula at many closely spaced points so we can see its curve.

On the 2025 training data, the exponential model has a lower mean squared error: about **170**, compared with about **312** for the line. That means it fits these nine examples more closely under this measure. But it does not prove that the exponential model will predict future prices better.

10:30–11:30 | Why time alone is flawed

[“Why this stock-price model is flawed.”]

So is this model capturing the true pattern that connects stock price with time? Is the exponential model more accurate?

Stock prices depend on many factors: company performance, market trends, economic conditions, investor expectations, and unexpected events. Our models only see time. They don’t know whether the company has a strong quarter, whether the market has changed, or whether a major product has been introduced.

That’s why this is not a dependable stock-forecasting method. By employing this method, you are basically betting that the company performance, market trend, etc will become better as time progresses, which is often not the case.

11:30–13:30 | Test on three unseen months

[Show the holdout table and the focused three-month graph.]

Now we test generalization. We fit both models using the 2025 months, then compare their predictions with the next three months in the 2026 file. That file restarts its local month index at zero, so we shift those inputs to continue the timeline: January, February, and March become month indices 9, 10, and 11.

We can see that, in March, the actual measure is about **$304.89**. The linear model predicts about **$350.24**, while the exponential model predicts about **$386.38**. The exponential estimate is more than **$81** away.

Across the three months, the linear model's average absolute error is about **$20.38**. The exponential model is about **$33.33**. So even though the exponential model had the lower training loss, it performed worse on these unseen months.

[Show the full-timeline graph directly below.]

The close-up helps us see the three test months. In the march of 2026, for example, there might be a shift in Google's operation strategy, or a new competitor just emerged. Obviously, it would be impossible to know that these would appear only by looking at time, and therefore the stock price cannot be accurately predicted by price. When doing scientific computing, it's critical to choose the model that actually discovered the **relationship**, instead of the one that minimized the **training loss**.

13:30–15:00 | Connect this to neural networks

["From simplicity to complexity."]

This example gave the model only one input: time. A more realistic model would need relevant information about the company and the market. We can think of those as additional X features.

A neural network combines information from multiple inputs through layers of calculations. During training, it adjusts internal values so the predictions fit examples. Because of its internal complexity, it's able to represent the relationship better with something like a linear regression, which, with only the slope and intercept, can only represent a fixed linear pattern.

That's our starting point: machine learning uses examples to fit patterns, and we judge those patterns by how well they generalize. In later meetings, we'll build the pieces of this process. We will talk about more examples of classification, supervised v.s. unsupervised learning in the future.

Thanks, everyone. Let's hand the time to Rian, who has years of experience in the machine learning industry.

[Closing]